Programmation sur Processeur Graphique

—

Partie 5 : mémoire partagée et synchronisation

Pierre-Emmanuel Hladik
pierre-emmanuel.hladik@ec-nantes.fr

11 mai 2025

Comment bien utiliser la mémoire du GPU

Pour apprendre à utiliser efficacement les types de mémoire CUDA dans un programme parallèle, il faut :

- comprendre l'importance de l'efficacité de l'accès à la mémoire
- connaître la distinction entre registres, mémoire partagée et mémoire globale
- comprendre la portée et la durée de vie des variables

Exemple d'accès mémoire (produit matriciel)

```
__global__
void sgemm(float *A, float *B, float *C,
                int numArows, int numAcolumns,
                int numBrows, int numBcolumns) {

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < numArows && col < numBcolumns) {
        float sum = 0;
        for (int n = 0; n < numAcolumns; n++) {
            sum += A[row * numAcolumns + n] * B[n * numBcolumns + col];
        }
        C[row * numBcolumns + col] = sum;
    }
}
```

Performance du GPU

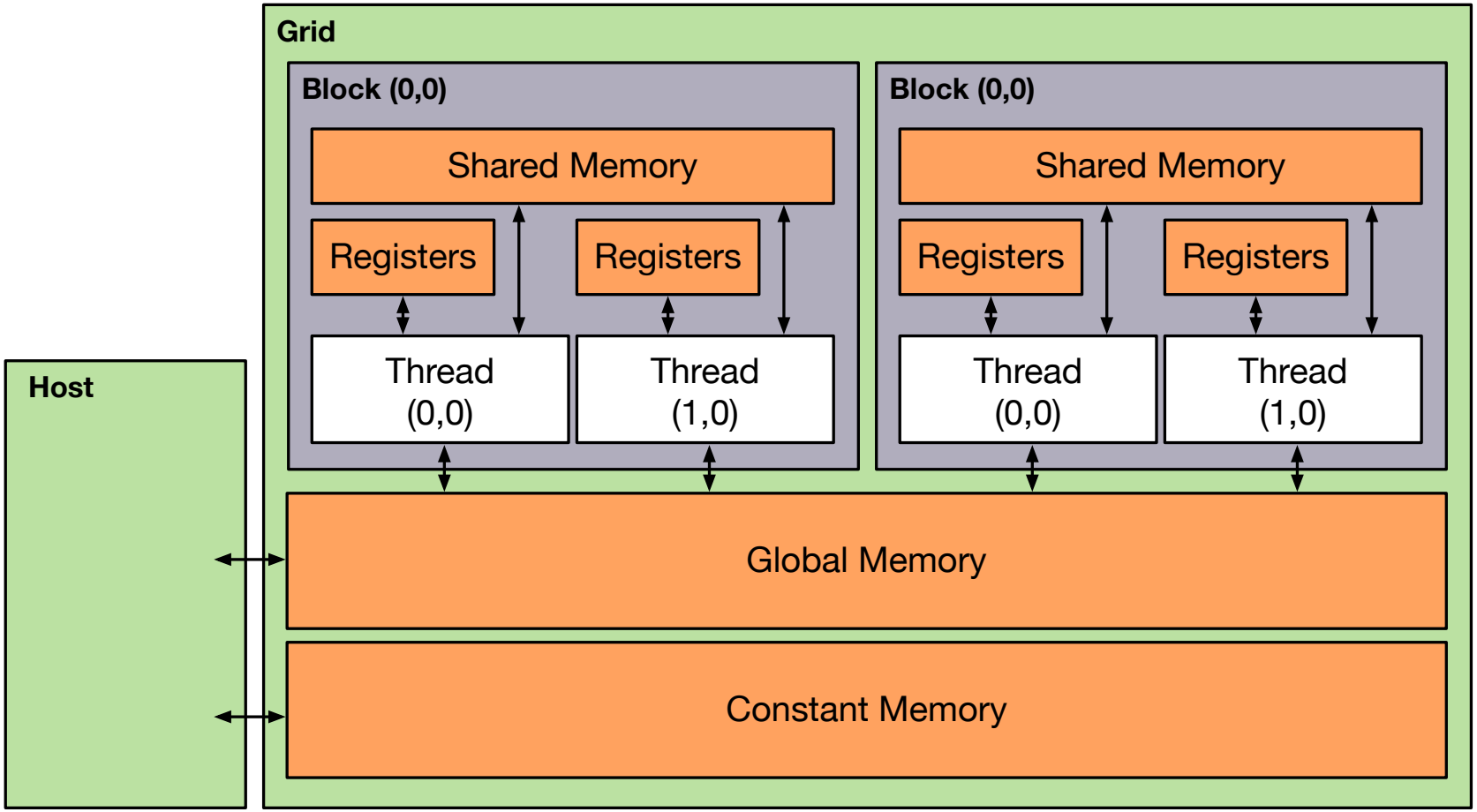
- Pour la multiplication `A[row * numAcolumns + n] * B[n * numBcolumns + col]`, tous les threads ont besoin de deux accès à la mémoire (2*4 octets)
- La Jetson Nano dispose d'une capacité de calcul de 472 GFLOPs (Floating-point Operations per Second)
- Si on veut exploiter au maximum la capacité de calcul, Il faut donc une bande passante de 8*472 = 3776 Go/s

Performance du GPU

- Pour la multiplication `A[row * numAcolumns + n] * B[n * numBcolumns + col]`, tous les threads ont besoin de deux accès à la mémoire (2*4 octets)
- La Jetson Nano dispose d'une capacité de calcul de 472 GFLOPs (Floating-point Operations per Second)
- Si on veut exploiter au maximum la capacité de calcul, Il faut donc une bande passante de 8*472 = 3776 Go/s

- La Jetson Nano a une mémoire de 4 GB avec une bande passante de 25.6 Go/s, soit pour 8 octets une limite de 3.2 FLOPs (0.6% du maximum) !
- Il faut réduire considérablement les accès à la mémoire pour se rapprocher des 472 GFLOPS

Mémoire CUDA vue par le programmeur



Focus *shared memory*

La *shared memory* est une mémoire dont le contenu est explicitement déclaré et utilisé dans le code source d'un *kernel*.

Elle a les caractéristiques suivantes :

- **Accès à une vitesse beaucoup plus élevée (en termes de latence et de débit) que la mémoire globale**
- Présente dans chaque Streaming Multiprocessor (SM), i.e.unité de calcul
- Portée limitée aux threads qui compose un block
- Durée de vie du block, i.e. le contenu disparait une fois que les threads terminent leur exécution
- Une forme de mémoire de type "scratchpad"
- Taille restreinte — Jetson Nano : "Maximum amount of shared memory per SM 64 KB" et "Maximum amount of shared memory per thread block 48 KB"

Déclaration des variables en CUDA pour un kernel

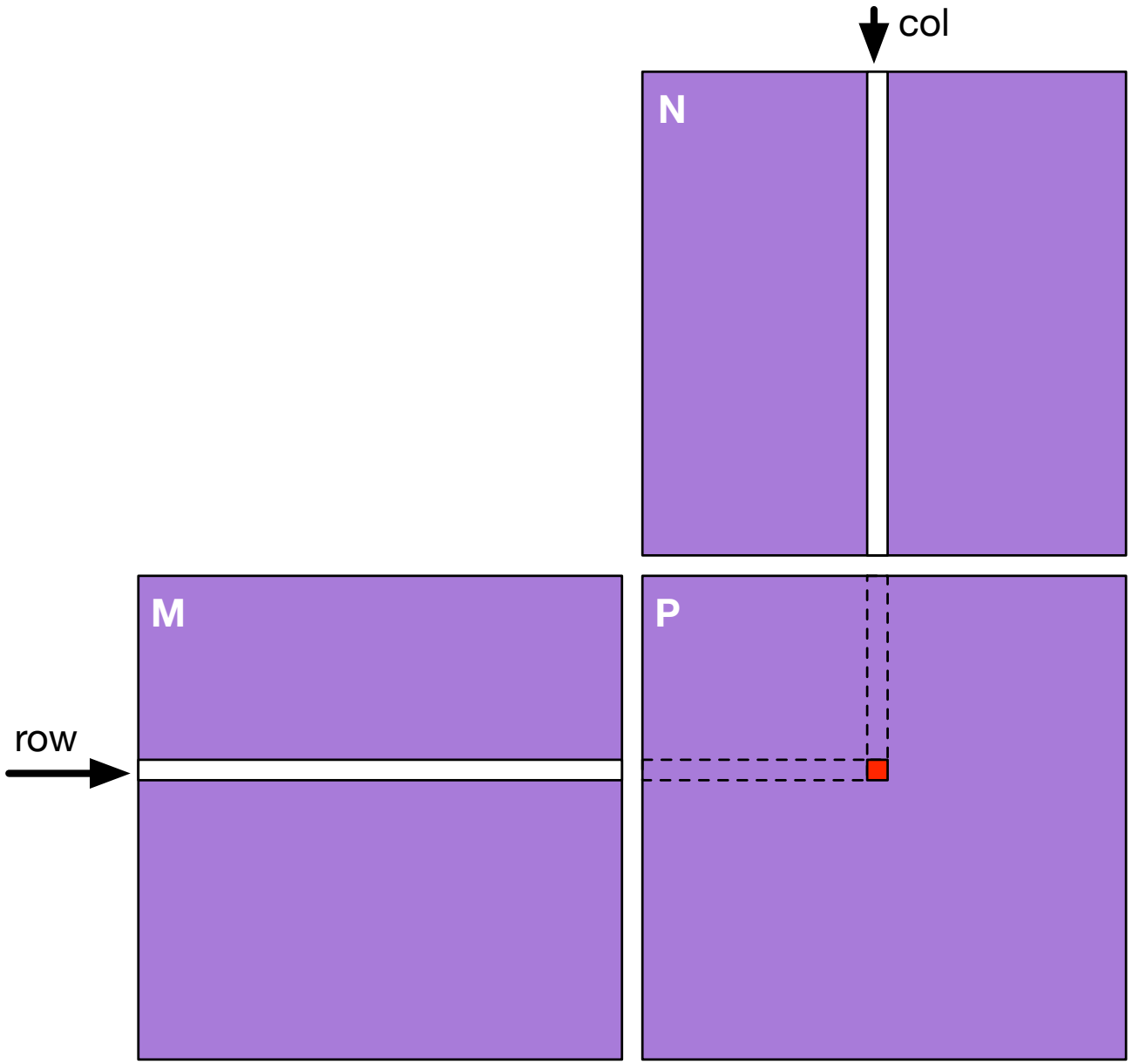
		Mémoire	Portée	Durée de vie
	`int LocalVar;`	registre	thread	thread
`__device__ __shared__`	`int SharedVar;`	shared	block	block
`__device__`	`int GlobalVar;`	global	grid	application
`__device__ __constant__`	`int ConstantVar;`	constant	grid	application

- `__device__` est optionnel avec `__shared__` et `__constant__`

Exemple

```
__shared__ int vector[NB_ELT];
__shared__ float ds_in[tileSize][tileSize];
```

Exemple d'utilisation : produit matriciel

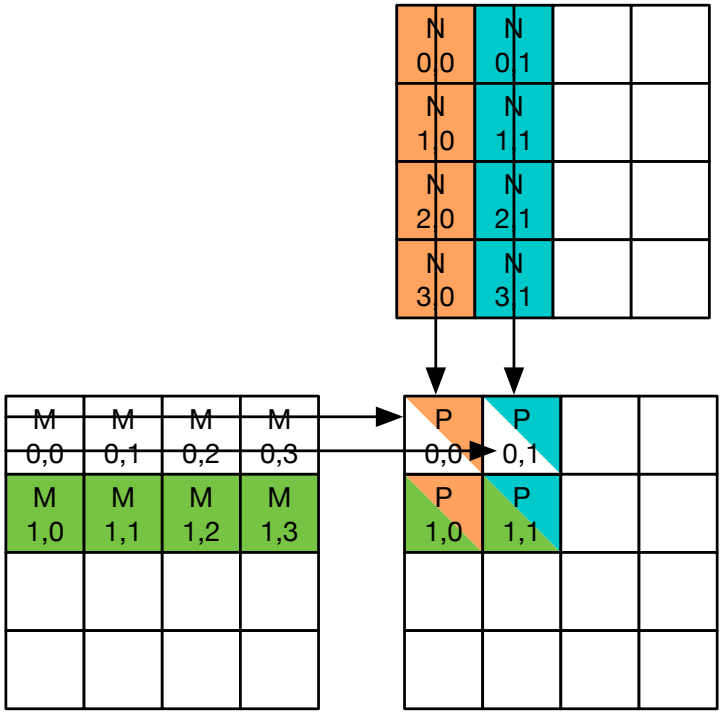


Kernel pour la multiplication matricielle : version naïve

```
/**
 Naive Kernel for matrix product P = M*N
 To simplify writing, matrices are assumed to be square (size given by width)
*/

__global__
void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  // Calculate the row index of the P element and M
  int row = blockIdx.y*blockDim.y+threadIdx.y;

  // Calculate the column index of P and N
  int col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((row < width) && (col < width)) {
    float sum = 0;
    // each thread computes one element of the block sub-matrix
    for (int k = 0; k < width; k++)
    {
      sum += M[row * width + k]*N[k * width + col];
    }
    P[row*width+col] = sum;
  }

}
```

- Chaque Thread `(row,col)` calcule $P_{row,col} = \sum_{k=0..width-1} M_{row,k} N_{k,col}$

Accès mémoire

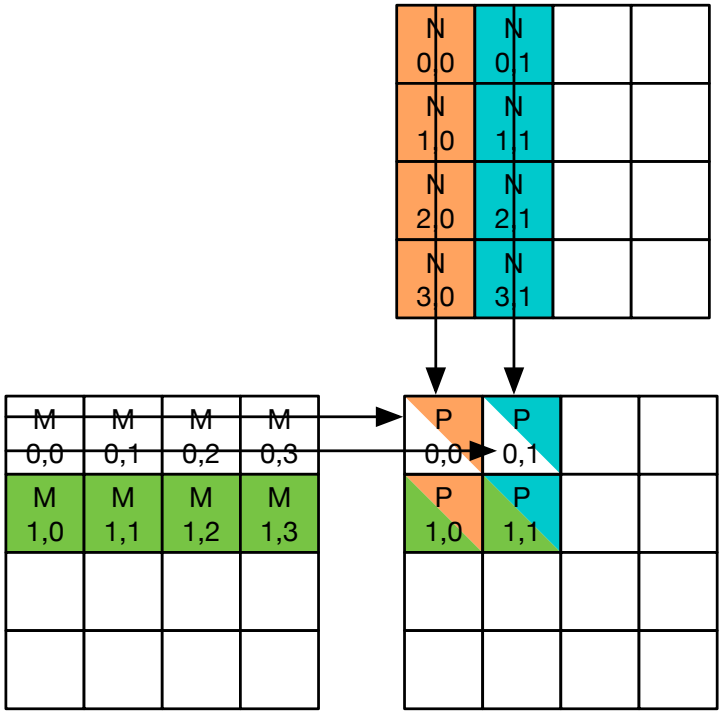


$\text{thread}_{0,0}$	$M_{0,0} * N_{0,0}$	$M_{0,1} * N_{1,0}$	$M_{0,2} * N_{2,0}$	$M_{0,3} * N_{3,0}$
$\text{thread}_{0,1}$	$M_{0,0} * N_{0,1}$	$M_{0,1} * N_{1,1}$	$M_{0,2} * N_{2,1}$	$M_{0,3} * N_{3,1}$

...

$\text{thread}_{0,j}$	$M_{0,0} * N_{0,j}$	$M_{0,1} * N_{1,j}$	$M_{0,2} * N_{2,j}$	$M_{0,3} * N_{3,j}$

Accès mémoire



$\text{thread}_{0,0}$	$M_{0,0} * N_{0,0}$	$M_{0,1} * N_{1,0}$	$M_{0,2} * N_{2,0}$	$M_{0,3} * N_{3,0}$
$\text{thread}_{1,0}$	$M_{1,0} * N_{0,0}$	$M_{2,1} * N_{1,0}$	$M_{3,2} * N_{2,0}$	$M_{4,3} * N_{3,0}$

...

$\text{thread}_{i,0}$	$M_{i,0} * N_{0,0}$	$M_{i,1} * N_{1,0}$	$M_{i,2} * N_{2,0}$	$M_{i,3} * N_{3,0}$

Utilisation de la mémoire partagée : produit matriciel

- **Principe :** Pré-charger les données dans la mémoire partagée (1 seul accès mémoire par donnée) puis on réalise les calculs

Utilisation de la mémoire partagée : produit matriciel

- **Principe :** Pré-charger les données dans la mémoire partagée (1 seul accès mémoire par donnée) puis on réalise les calculs
- **Problème :** la mémoire partagée est limitée (64 ko au total ou 48ko/block), si les matrices sont grandes il n'est pas possible de tout pré-charger. Pour une donnée de 4 octets, vous pouvez seulement avoir 12000 valeurs par block. Dans notre cas cela correspond à deux matrices de 77x77. S'il y a quatre blocks, la limite est de 16ko par block, soit des matrices de 63x63.
- **Solution :** découpage en tuiles

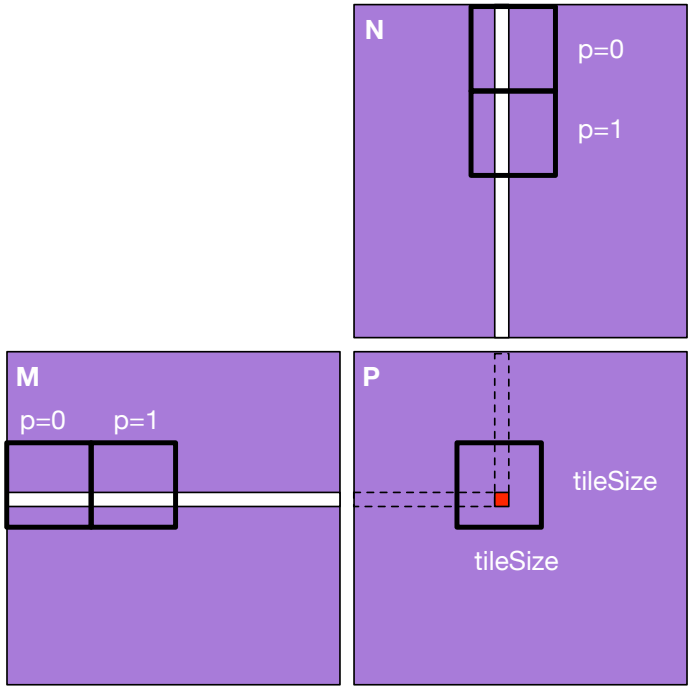
Principe de la multiplication matricielle par tuile

On utilise le fait que (sans les tests au bord) :

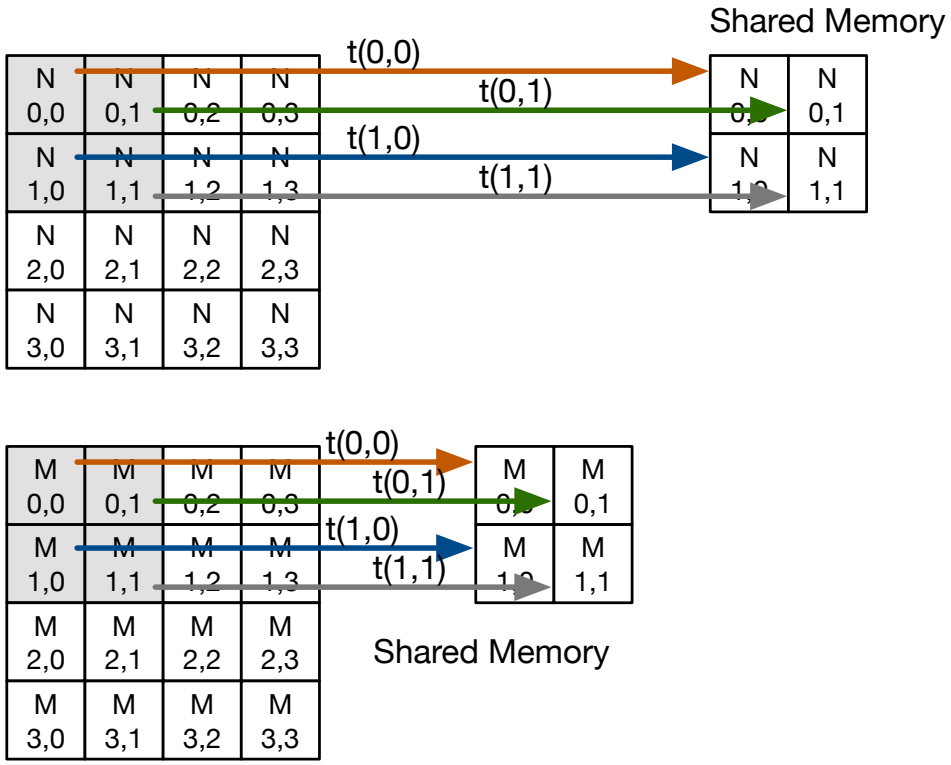
$$
\begin{aligned}
P_{row,col} &= \sum_{k=0}^{width-1} M[row, k] \times N[k, col] \\
&= \sum_{p=0}^{\frac{width-\mathbf{1}}{tileSize}} \sum_{k=0}^{tileSize} M[row, p * tileSize + k] \times N[p * tileSize + k, col]
\end{aligned}
$$

Multiplication matricielle par tuile
Tiled multiplication matrix

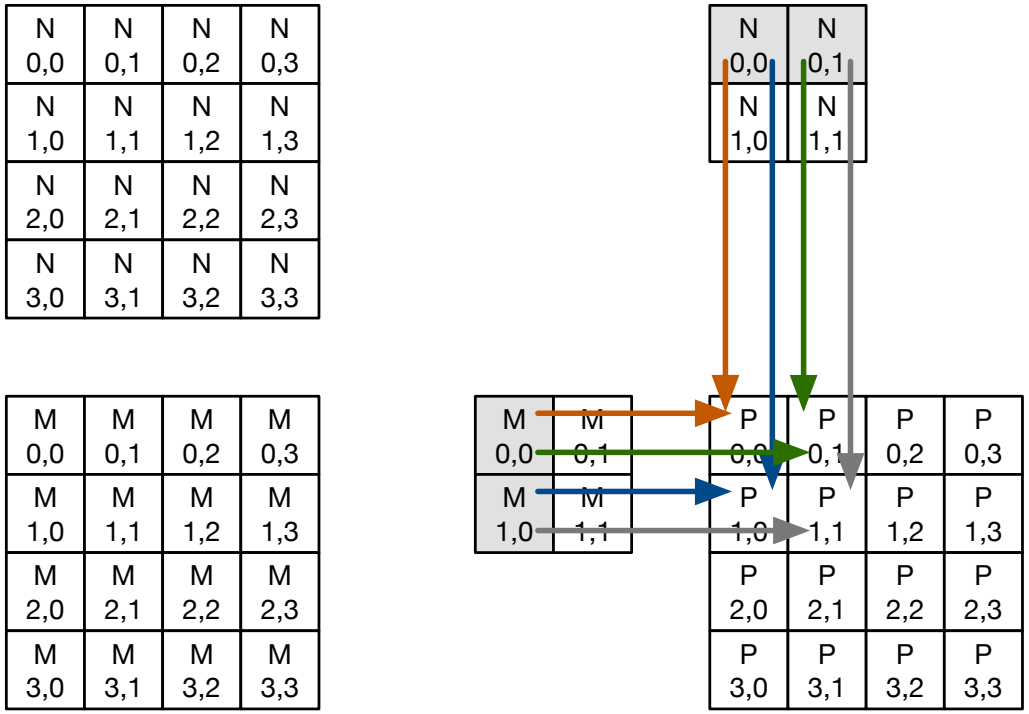
- Chaque thread réalise le calcul d'un élément de la matrice produit P
- Les threads sont regroupés par block de taille `tileSize` $\times$ `tileSize`
- Le calcul est divisé en `p` étapes qui réalisent :
  - Le chargement en mémoire partagée d'une tuile de M et d'une de N de taille `tileSize` $\times$ `tileSize`
  - La somme des produits de ces sous-matrices



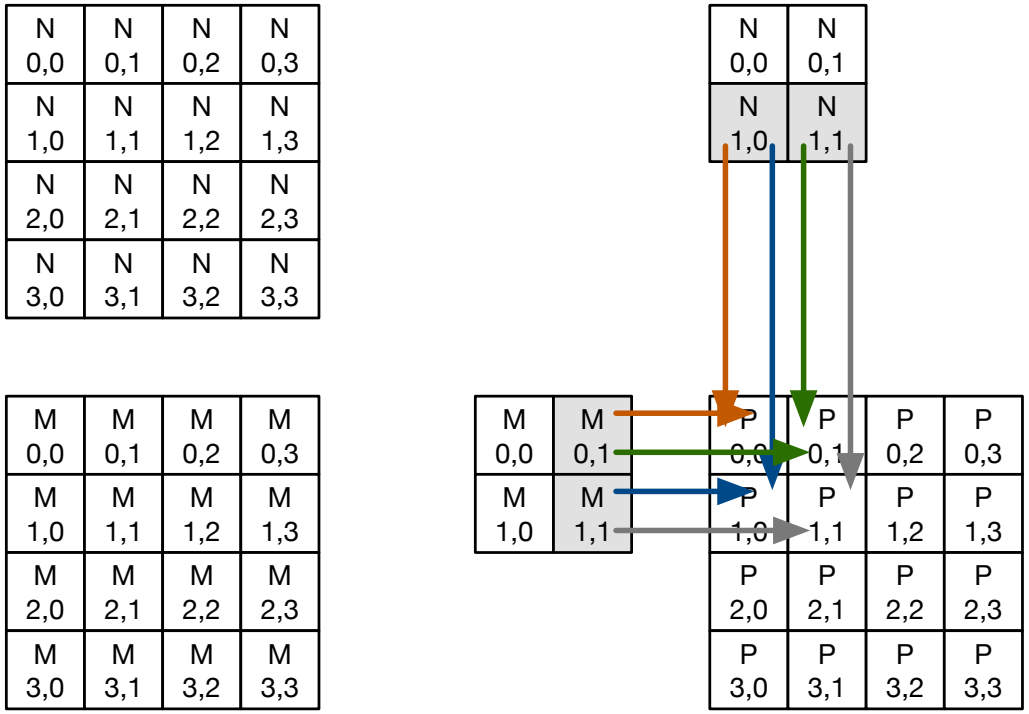
Multiplication matricielle par tuile (phase p=0)



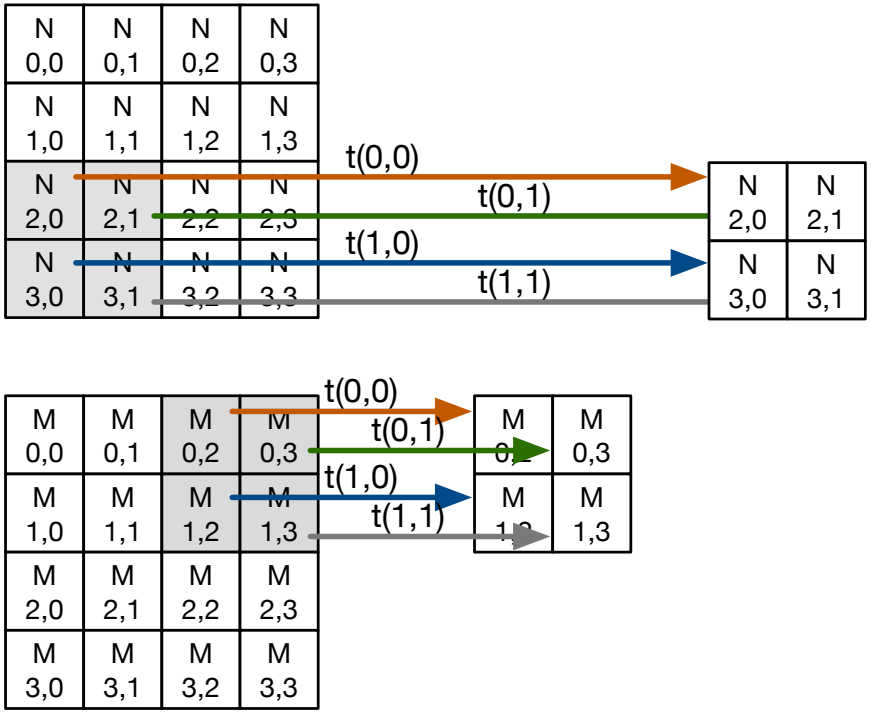
Multiplication matricielle par tuile (phase 0)



Multiplication matricielle par tuile (phase p=0)



Multiplication matricielle par tuile (phase p=1)

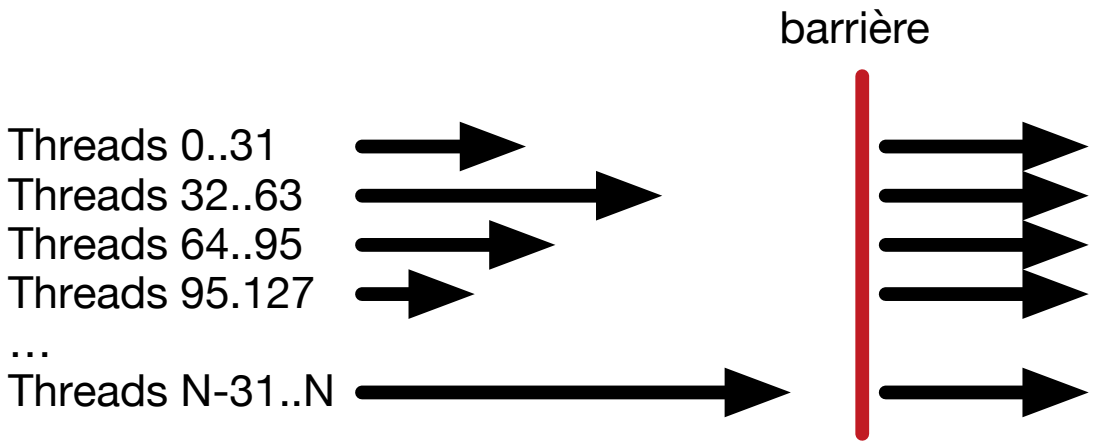


Code du kernel

```
#define tileSize 32

__global__
void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  // Defining shared memory workspaces
  __shared__ float ds_M[tileSize][tileSize];
  __shared__ float ds_N[tileSize][tileSize];

  // Redefinition of blockIdx.x, blockIdx.y,
  // threadIdx.x and threadIdx.y to simplify writing
  int bx = blockIdx.x; int by  = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  // Definition of row and col
  int row = by * blockDim.y + ty;
  int col = bx * blockDim.x + tx;

  // intermediate variable for calculating the sum of products
  float sum = 0.0;

  // Loop over the M and N tiles required to compute the P element
  for (int p = 0; p < width / tileSize; p++) {
    // collaborative loading of M and N tiles into shared memory
    ds_M[ty][tx] = M[row * width + p * tileSize + tx];
    ds_N[ty][tx] = N[(p * tileSize + ty) * width + col];

    for (int k = 0; k < tileSize; k++)
    {
        sum+= ds_M[ty][k] * ds_N[k][tx];
    }
  }
  P[row * width + col] = sum;
}
```

Attention dangers : synchronisation

Que se passe-t-il si un thread (d'un autre warp) n'a pas chargé la mémoire avant de commencer les calculs ?




```
// synchronize threads in this block
__syncthreads();
```

Garantit que chaque **thread du block** a terminé ses instructions après l'appel à `__syncthreads()`

Exemple avec la multiplication matricielle (carré) : kernel

```
__global__
void MatrixMulKernel(float* M, float* N, float* P, int width)
{
  __shared__ float ds_M[tileSize][tileSize];
  __shared__ float ds_N[tileSize][tileSize];

  int bx = blockIdx.x; int by  = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  int row = by * blockDim.y + ty;
  int col = bx * blockDim.x + tx;
  float sum = 0.0;

  for (int p = 0; p < width / tileSize; p++) {
    ds_M[ty][tx] = M[row * width + p * tileSize + tx];
    ds_N[ty][tx] = N[(p * tileSize + ty) * width + col];

     // Synchronization to ensure that shared memory has been
     // fully loaded by all threads before starting calculations
    __syncthreads();

    for (int i = 0; i < tileSize; ++i)
    {
        sum+= ds_M[ty][i] * ds_N[i][tx];
    }

   // Synchronization to ensure that shared memory is not
   // modified before calculations are completed for all threads
    __syncthreads();
  }
  P[row * width + col] = sum;
}
```

Considérations sur la taille des tuiles (block de threads)

- Supposons deux valeurs pour `tileSize` :
  - 16 : ce qui donne $16 \times 16 = 256$ threads par block
  - 32 : ce qui donne $32 \times 32 = 1024$ threads par block
- Pour 16, à chaque phase, chaque block effectue $2 \times 256 = 512$ chargements de flottants depuis la mémoire globale pour $256 \times (2 \times 16) = 8192$ opérations mul/add, soit 16 opérations à virgule flottante pour chaque chargement de mémoire
- Pour 32, dans chaque phase, chaque block effectue $2 \times 1024 = 2048$ chargements de flottants depuis la mémoire globale pour $1024 \times (2 \times 32) = 65536$ opérations mul/add, soit **32 opérations à virgule flottante pour chaque chargement de mémoire**

Shared Memory et Threading

Lien : compute capabilities (SM avec 64 Ko de mémoire partagée)

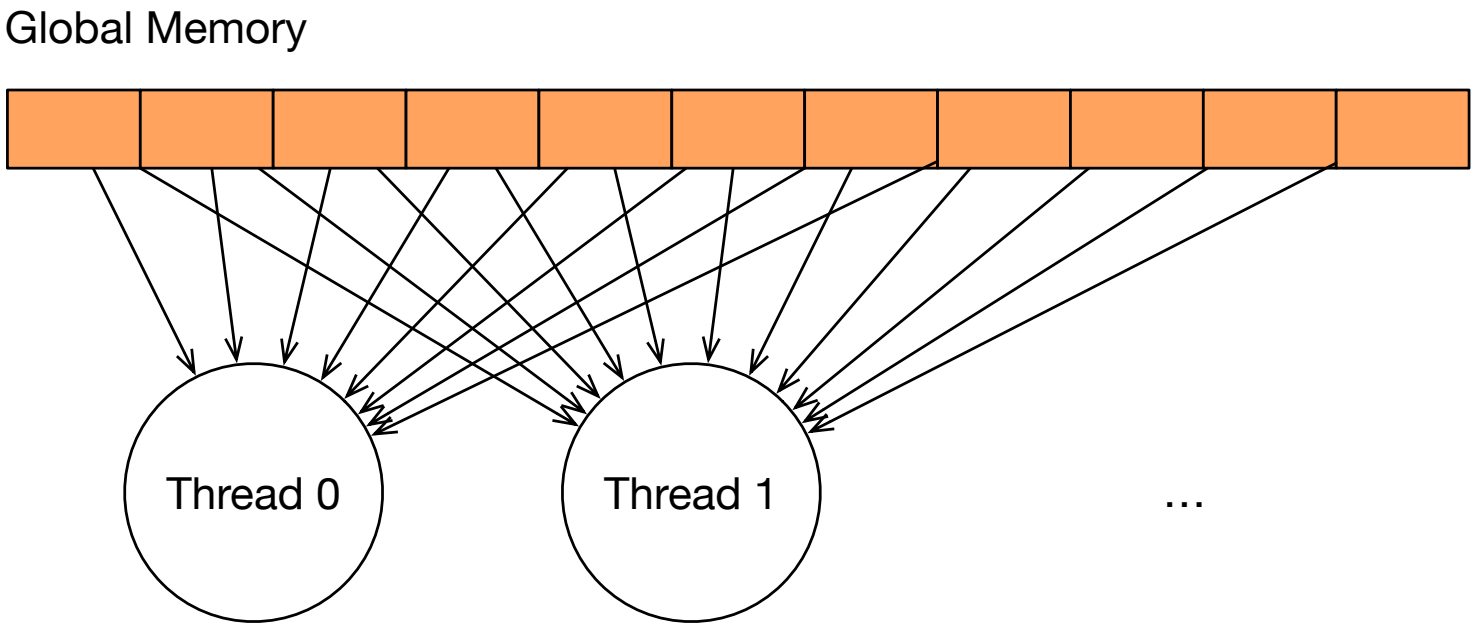
- La taille de la mémoire partagée dépend de l'implémentation !
- Pour `tileSize`=16, chaque block de threads utilise 2*256*4o=2Ko de mémoire partagée
  - Pour une mémoire partagée de 64 Ko, on peut potentiellement avoir jusqu'à 32 blocks de threads qui exécutent des opérations
  - Cela permet jusqu'à 32*512 = 16 384 chargements mémoire simultanés (2 par thread, 256 threads par block)
- Pour tileSize=32, chaque block de thread utilise 2*32*32*4o=8Ko de mémoire partagée
  - Pour une mémoire partagée de 64 Ko, on peut potentiellement avoir jusqu'à 8 blocks de threads qui exécutent des opérations
  - Cela permet jusqu'à 8*2048 = 65 536 chargements mémoire simultanés (2 par thread, 1024 threads par block)
- Chaque __syncthread() peut réduire le nombre de warps actifs pour un block : un grand nombre de blocks peut être avantageux

Généralisation : technique de décomposition en tuiles

Si plusieurs threads font fréquemment des accès à des données communes, on découpe l'algorithme de traitement en plusieurs phases. A chaque phase :

1. on charge les données de la mémoire globale dans une mémoire partagée, puis
2. on réalise les calculs sur ce sous-ensemble de valeurs.

L'objectif est de réduire l'effet limitatif de la bande passante mémoire sur les performances des kernels.

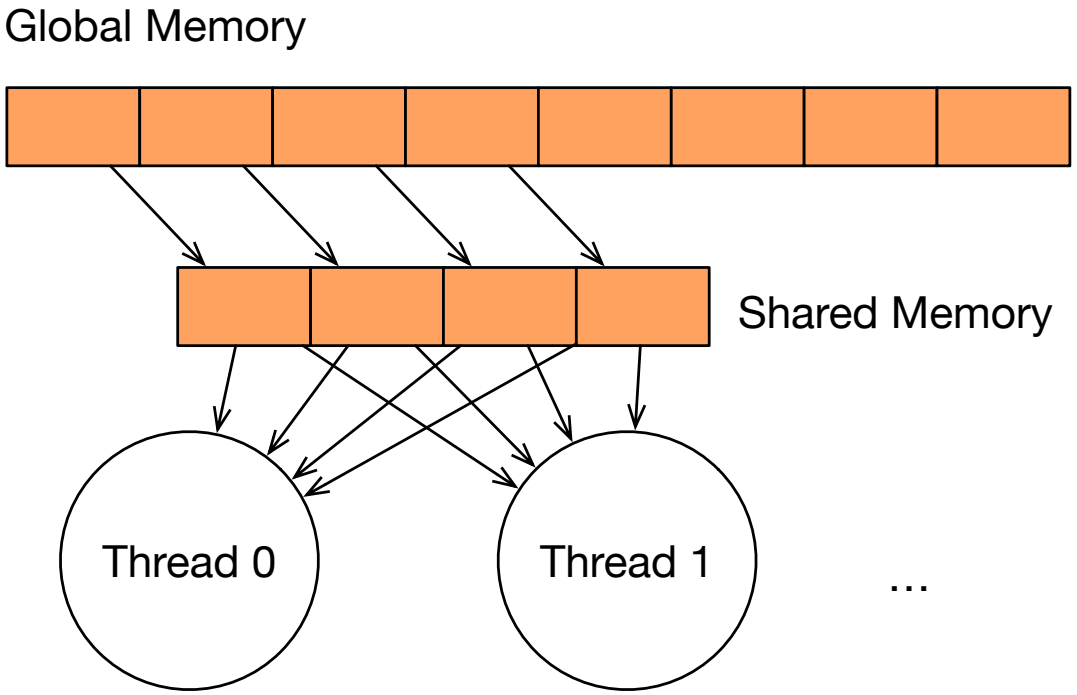


Généralisation : technique de décomposition en tuiles

Si plusieurs threads font fréquemment des accès à des données communes, on découpe l'algorithme de traitement en plusieurs phases. A chaque phase :

1. on charge les données de la mémoire globale dans une mémoire partagée, puis
2. on réalise les calculs sur ce sous-ensemble de valeurs.

L'objectif est de réduire l'effet limitatif de la bande passante mémoire sur les performances des kernels.

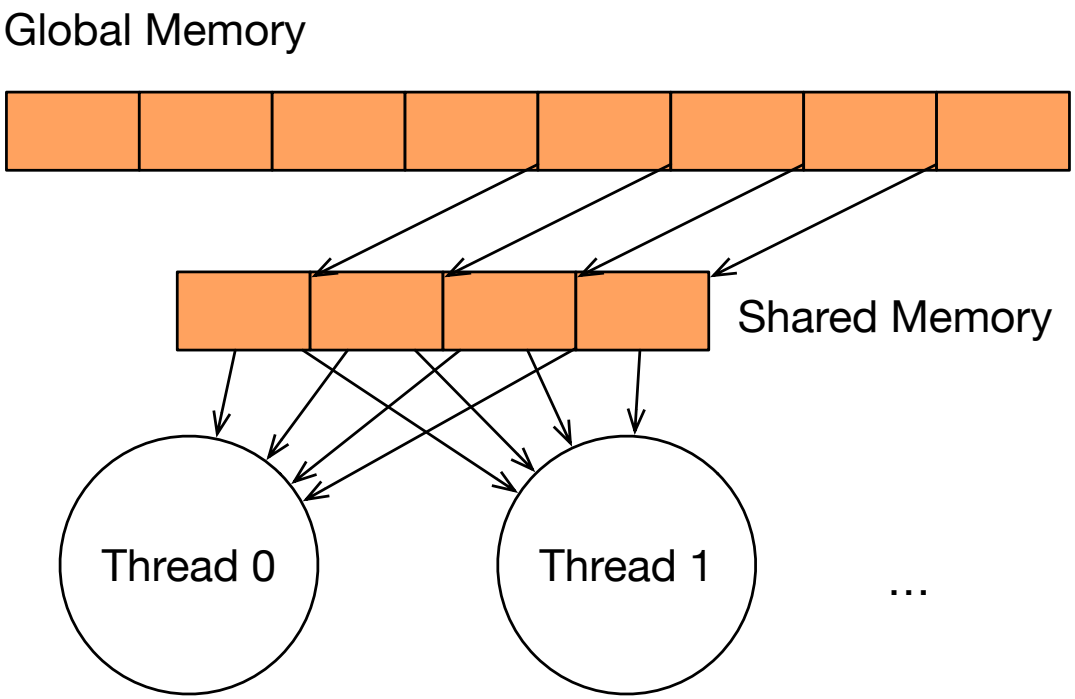


Généralisation : technique de décomposition en tuiles

Si plusieurs threads font fréquemment des accès à des données communes, on découpe l'algorithme de traitement en plusieurs phases. A chaque phase :

1. on charge les données de la mémoire globale dans une mémoire partagée, puis
2. on réalise les calculs sur ce sous-ensemble de valeurs.

L'objectif est de réduire l'effet limitatif de la bande passante mémoire sur les performances des kernels.



Retour sur la multiplication de matrices : gérer la taille des matrices

- **Problème :** Que faire pour les valeurs dans une tuile qui est en dehors de la matrice ?
- **Solution :** on charge une valeur de 0.0 en mémoire partagée

```
__global__
void MatrixMulKernel(float* M, float* N, float* P, int width) // square matrix
{
  ...
  for (int p = 0; p < (width-1)/tileSize + 1; ++p) {
    if(row < width && ty * tileSize+tx < width)
        { ds_M[ty][tx] = M[row*width + p*tileSize+tx]; }
    else
        { ds_M[ty][tx] = 0.0; } // Load default value 0.0
    if (p*tileSize+ty < width && col < width)
        { ds_N[ty][tx] = N[(t*tileSize+ty)*width + col]; }
    else
        { ds_M[ty][tx] = 0.0; } // Load default value 0.0
    __syncthreads();

    if(row < width && col < width) {
        for (int i = 0; i < tileSize; ++i)
                { sum+= ds_M[ty][i] * ds_N[i][tx]; }
    }
    __syncthreads();
  }
  if (row < width && col < width)
          { P[row*width+col] = sum; }
}
```

Un peu de pratique avec le TD 5

Objectifs

- Utiliser des mécanismes de synchronisation
- Utiliser la mémoire partagée
- Mettre en place une décomposition en tuile